NOTIFYHUB

Centralized Digital Notice & Communication Platform

“Never miss a campus update”

Software Requirements, Architecture & Development Documentation

Prepared by: Koppula Yashwanth Reddy

Document Version: 1.0

Date: 3 August 2026

1. Abstract

1.1 Justification for the Proposed Digital Solution

Educational institutions rely on timely, accurate communication to function well. Notices about examinations, fee deadlines, placement drives, faculty meetings, and emergency alerts all need to reach the right audience quickly and reliably. In most colleges and universities today, this is still handled through physical notice boards, WhatsApp groups, email chains, or a patchwork of departmental circulars. As an institution grows in size and complexity — multiple faculties, departments, and hundreds of staff and students — this informal approach breaks down. Notices get missed, duplicated, or reach students after the relevant deadline has passed.

NotifyHub is proposed as a single, centralized, role-aware communication system that gives every stakeholder in the institution — from the Super Admin down to individual students — a clear, structured channel to publish, discover, and respond to information. The justification for building it rests on three pillars: (a) reducing the time-to-communicate for urgent notices from hours/days to minutes, (b) providing accountability and traceability of who published what and when, and (c) giving students a direct, monitored channel to raise queries to the administration with an enforced response-time SLA (target: under 6 hours).

1.2 Analysis of Existing Notice Board Systems

Existing communication mechanisms in most institutions fall into two broad categories, both of which have well-documented shortcomings:

1.2.1 Traditional (Physical) Notice Boards

- No real-time delivery — a notice posted on a board is only visible to someone who physically walks past it.
- No record of who saw the notice, whether it was read, or when it was taken down.
- No way to target a notice at a specific department, faculty, or year group without printing multiple copies.
- No search or history — once removed, a past notice is effectively lost.
- No mechanism for a student to ask a follow-up question about a notice.
- Physically constrained — limited board space forces older notices to be removed prematurely.

1.2.2 Ad-hoc Digital Alternatives (Email, WhatsApp, Institutional Portals)

- Email is easily missed amid unrelated traffic and has no urgency differentiation.
- WhatsApp groups are informal, hard to moderate, not role-based, and not auditable.
- Generic institutional portals are often static content-management systems with no live alerting, no query routing, and no distinction between an announcement, an event, and an urgent alert.
- None of the above enforce a response-time commitment for student queries, and none provide a real-time ‘live counter’ for time-sensitive notices (e.g., registration closing in X hours).

1.3 How NotifyHub Addresses These Shortcomings

NotifyHub is designed specifically to close the gaps identified above:

- A role-based hierarchy (Super Admin → Principal → Dean → HoD → Faculty → Student) ensures every notice has a clear, authorized publisher and a defined audience.
- Announcements, Events, and Alerts are modeled as distinct content types, with Alerts visually flagged in red and supported by a live countdown/urgency counter.
- A built-in search and filter engine lets any user quickly locate a notice by keyword, department, category, or date instead of scrolling a physical board or a chat history.
- A structured Query module lets students raise questions directly to the Super Admin, with the system tracking and enforcing a target response time of under 6 hours.
- Secure, hashed authentication (bcrypt) and strict role-based authorization ensure only the correct users can create or edit content, and that only the Super Admin can ever delete a notice once it has been published.
- A webhook-driven, real-time architecture ensures new announcements and alerts propagate to connected clients immediately, rather than requiring a manual page refresh.

In summary, NotifyHub converts a fragmented, unaccountable, and delay-prone communication process into a single source of truth: fast, searchable, auditable, and tailored to the institution's organizational structure.

2. Stakeholders & Role-Based Access Control (RBAC)

NotifyHub's authorization model is the backbone of the system. Every module — announcements, events, alerts, account management, and queries — checks the requester's role before allowing an action. The system is intentionally kept to a single administrative role at the top: there is no separate ‘Admin’ role. The Super Admin alone carries full administrative authority, exercised through a dedicated Account Manager module, and is the only role in the system that can delete a notice once it has been posted.

Role	Scope	Key Permissions
Super Admin	Entire institution	Full system control through a dedicated Account Manager: create, update, deactivate, and manage every user account across all roles; reset any user's password; publish institution-wide content; the only role authorized to delete a notice once it has been published; view all queries; view system-wide analytics and audit logs; override any content.
Principal	Institution-wide communications	Publish and edit institution-wide announcements & alerts; oversee Deans' and HoDs' communications; view aggregated reporting. Cannot delete a notice once it is published — only the Super Admin can.
Dean	Faculty-level	Publish/edit notices and events for their faculty; oversee HoDs and Faculty members within the faculty; view faculty-level query analytics. No delete access on published content.
HoD	Department-level	Publish/edit notices for their department; review Faculty-authored notices; respond to escalated student queries. No delete access on published content.
Faculty	Course/section-level	Post and edit notices/events relevant to their courses; respond to student queries directly. No delete access on published content.

Role	Scope	Key Permissions
Student	Read + Query	View announcements, events, and alerts (filtered/searchable); raise queries to the Super Admin; track the status and response history of their own queries.

2.1 The Super-Admin-Only Delete Rule

A key business rule governing NotifyHub is that once a notice — an Announcement, Event, or Alert — has been published, it can only be deleted by the Super Admin. Every other role, including the original author (Principal, Dean, HoD, or Faculty), retains the ability to edit their own content but has no delete capability once it goes live. This design choice centralizes accountability for content removal, prevents accidental or unilateral takedowns of institution-facing information, and keeps a single, auditable point of control over what disappears from the platform.

2.2 Account Manager (Super Admin)

All account and role administration in NotifyHub is consolidated under the Super Admin's Account Manager module. Through it, the Super Admin can create new accounts for any role, update or deactivate existing accounts, change a user's assigned role or scope (department/faculty), and reset any user's password. Every action performed through the Account Manager is written to an append-only audit log capturing the actor, the target account, the action, and a timestamp.

3. Requirements Overview

3.1 Functional Requirements

- The system shall support role-based login and authorization for all six user roles: Super Admin, Principal, Dean, HoD, Faculty, and Student.
- The system shall allow authorized roles (Super Admin, Principal, Dean, HoD, Faculty) to create and edit Announcements, Events, and Alerts within their scope.
- The system shall restrict the deletion of a published notice exclusively to the Super Admin — no other role, including the original author, may delete a notice once it is live.
- The system shall visually distinguish Alerts/Urgent notices using a red color scheme and shall support a live countdown/counter for time-bound alerts.
- The system shall provide keyword search and category/department/date filters across all notice types.
- The system shall allow Students to submit queries to the Super Admin, and shall track the elapsed time against a target response SLA of under 6 hours.
- The Super Admin shall have access to a dedicated Account Manager to create, update, deactivate, and manage every account in the system, and to reset any user's password.
- The Home page shall display a live count of total notices and upcoming events, and a short explainer on why NotifyHub exists.
- The system shall expose a webhook endpoint so that new announcements/alerts can trigger downstream notifications (e.g., push notification services, email digests, Slack/Teams relays).

- The system shall maintain a Contact page for reaching institutional administrators.
- All passwords shall be stored using one-way bcrypt hashing; plaintext passwords shall never be persisted or logged.

3.2 Non-Functional Requirements

- Performance: Home page and notice list should render within 2 seconds on a standard broadband/campus Wi-Fi connection.
- Scalability: The system should comfortably support a multi-faculty university (thousands of students, hundreds of staff) without redesign.
- Availability: Target uptime of 99.5%, consistent with a Vercel-hosted serverless deployment.
- Security: Role-based authorization on every API route, with the delete operation on notices additionally gated to the Super Admin alone; hashed credentials; input validation and sanitization on all forms.
- Usability: Mobile-first, responsive UI built with Tailwind CSS so it works equally well on phones (where most students will check notices) and desktops (where staff will publish them).
- Auditability: Every content create/update action and every Account Manager action is logged with actor, timestamp, and target; every delete action (Super Admin only) is logged with the same detail.
- Maintainability: Clear separation of concerns between frontend, backend, and database layers to allow independent updates.

4. Technology Stack & System Architecture

4.1 Technology Stack

Layer	Technology	Purpose
Frontend	React.js, Tailwind CSS, Axios/Fetch	Component-based, responsive UI; API communication.
Backend	Node.js, Express.js	REST API layer, route handling, business logic, middleware (auth, validation).
Security	bcrypt	One-way password hashing for all stored credentials.
Database	PostgreSQL (Neon, serverless)	Relational storage for users, announcements, events, and queries.
Deployment / Hosting	Vercel — single combined deployment	Backend serves the built React frontend as static files from its own public folder; one project, one URL, no separate frontend hosting.
Version Control	Git & GitHub	Source control, branching strategy, pull-request based review.
Real-time layer	Webhooks (+ optional SSE/WebSocket)	Push new announcements/alerts to connected clients and external integrations.

4.2 High-Level Architecture

NotifyHub follows a three-tier web architecture, deployed as a single combined project for cost efficiency and operational simplicity:

- Presentation Tier: A React.js single-page application, styled with Tailwind CSS, responsible for rendering the five core pages (Home, Contact, Announcements, Events, Alerts) and the Super Admin dashboard.
- Application Tier: An Express.js REST API that also serves the built frontend as static files. Middleware chain: request logging → JWT auth → role authorization → input validation → controller → response.
- Data Tier: Neon serverless PostgreSQL, storing normalized tables for users/roles, announcements, events, and queries, with foreign keys enforcing department/faculty scoping.
- Integration Layer: A webhook dispatcher fires on notice creation/update, allowing downstream systems (push notification service, email digest job, Slack/Teams channel) to subscribe without coupling them to the core application.

4.3 Deployment Strategy — Single Combined Deployment

Rather than deploying the frontend and backend as two individual, separately hosted services (a static frontend deployment plus a separate API deployment), NotifyHub uses a single combined deployment: the Express backend builds and serves the React frontend directly from its own public folder. This removes the need for CORS configuration between frontend and backend (they share the same origin), collapses two deployment pipelines into one, and gives the whole application a single URL and a single environment-variable set to manage.

The deployment steps are:

```
# 1. Build the frontend
cd frontend
npm run build          # produces a /dist folder

# 2. Copy the build output into the backend's public folder
cp -r dist/* ../backend/public/

# 3. Remove any conflicting GET / route already defined in the
#    backend (e.g. a placeholder or health-check handler) —
#    it would otherwise shadow the frontend's index.html.

# 4. Serve the frontend as static files (Express, in app.js):
app.use(express.static("public"));

# 5. Deploy only the backend project to Vercel.
#    Hitting the root "/" route now serves the React app;
#    /api/* routes continue to be handled by Express as normal.
```

Because client-side routing (React Router) needs every non-API path to resolve to index.html, a catch-all fallback is added after all /api routes:

```
app.use(express.static("public"));

app.get("*", (req, res) => {
  res.sendFile(path.join(__dirname, "public", "index.html"));
});
```

This single-deployment pattern is the default for NotifyHub. A split (individual) deployment — frontend and backend hosted as two separate Vercel projects — remains an option if the team later needs to scale the frontend and API independently, but is not the initial approach.

4.4 Core Database Entities

Table	Key Columns	Notes
users	id, name, email, password_hash, role, department_id, faculty_id, created_at	role is an enum: super_admin, principal, dean, hod, faculty, student.
announcements	id, title, body, category, department_id, is_urgent, author_id, published_at, expires_at, deleted_at	Only a super_admin-authenticated request may set deleted_at / remove a row.
events	id, title, description, event_date, location, department_id, author_id, created_at, deleted_at	Powers the 'Upcoming Events' widget on the Home page.
queries	id, student_id, subject, message, status, assigned_to, created_at, responded_at	status tracks open/in_progress/resolved; responded_at − created_at is measured against the 6-hour SLA.
departments / faculties	id, name, parent_faculty_id	Provides the scoping hierarchy referenced by Principal/Dean/HoD permissions.

5. Application Pages & Feature Breakdown

The scope defines five core pages, plus the Super Admin dashboard:

5.1 Home Page

- Total notice count and upcoming events count, updated in real time.
- Short 'Why NotifyHub' explainer section for new/first-time users.
- Tagline: "Never miss a campus update."
- Quick links into Announcements, Events, and Alerts.

5.2 Announcements Page

- Chronological, filterable list of published notices.
- Search bar (keyword) plus filters for department, category, and date range.
- Role-gated create/edit controls visible only to authorized publishers (Principal, Dean, HoD, Faculty, Super Admin).
- The delete control is rendered only for the Super Admin — no other role ever sees or can trigger it, even via a direct API call.

5.3 Events Page

- Calendar/list view of upcoming institutional and departmental events.
- Each event links back to its originating department/faculty.

5.4 Alerts / Urgent Page

- Visually distinct red color scheme to signal urgency.
- Live countdown/counter for time-bound alerts (e.g., 'Registration closes in 04:12:00').
- Highest-priority placement — surfaced on the Home page banner as well.

5.5 Contact Page

- Directory of departmental/administrative contact points.
- Entry point into the Student Query workflow described below.

5.6 Query Workflow (Student ↔ Super Admin)

- Student submits a query from the Contact page or a dedicated 'Ask a Question' form.
- Query is timestamped and routed to the Super Admin (or, for department-level matters, escalated to the relevant HoD).
- System tracks elapsed time and flags queries approaching or exceeding the 6-hour response target.
- Student can view the status and response history of their own queries.

5.7 Account Manager (Super Admin Only)

- Create, update, deactivate, and manage every user account in the system, across all roles.
- Reset any user's password.
- Assign or change a user's role and department/faculty scope.
- Delete any published notice — the only role in NotifyHub with this capability.
- Full audit trail of every account-management and delete action.

6. Low-Level Architecture

This section captures the concrete, implementation-level view of NotifyHub: backend folder structure, the middleware request chain, and the complete route table with the role required for each endpoint.

6.1 Backend Folder Structure

```
backend/
├── public/               # built frontend assets (copied from frontend/dist)
├── src/
│   ├── config/           # db.js, env.js
│   ├── middleware/       # auth.js, requireRole.js, validate.js, errorHandler.js
│   ├── routes/           # auth, announcements, events, alerts, queries, users, webhooks
│   ├── controllers/      # request/response handling per route
│   ├── services/         # business logic (SLA calc, role rules, webhook dispatch)
│   ├── models/           # Prisma/Drizzle schema or raw SQL query modules
│   └── app.js            # Express app: middleware chain + route mounting
├── server.js             # entry point — starts the HTTP server
└── package.json

frontend/
├── src/
│   ├── pages/            # Home, Announcements, Events, Alerts, Contact
│   ├── components/
│   ├── dashboard/        # Super Admin — Account Manager UI
│   └── api/              # Axios/Fetch client wrappers
└── package.json
```

6.2 Middleware Chain (Request Lifecycle)

Every API request passes through the same ordered chain before it reaches business logic:

- 1. Request logger — records method, path, and timestamp.
- 2. JSON body parser.
- 3. JWT authentication — verifies the token and attaches req.user (id, role, department_id/faculty_id).
- 4. Role authorization — requireRole([...]) checks req.user.role against the route's allowed roles; delete routes hard-code requireRole(['super_admin']) with no exceptions.
- 5. Input validation — schema validation (Zod/Joi) of the request body/params.
- 6. Controller — calls the service layer and shapes the response.
- 7. Centralized error handler — catches thrown errors and returns a consistent error shape.

6.3 Complete Route Table

Method	Path	Role(s) Allowed	Description
POST	/api/auth/login	Public	Authenticate credentials (bcrypt compare) and issue a JWT.
POST	/api/auth/logout	Authenticated	Invalidate/clear the current session.
GET	/api/users	super_admin	List all accounts (Account Manager).
POST	/api/users	super_admin	Create a new account for any role.
PUT	/api/users/:id	super_admin	Update an account's details, role, or scope.
DELETE	/api/users/:id	super_admin	Deactivate/remove an account.
POST	/api/users/:id/reset-password	super_admin	Reset any user's password.
GET	/api/announcements	All roles (public read, filtered)	List/search/filter announcements.
POST	/api/announcements	principal, dean, hod, faculty, super_admin	Create a new announcement.
PUT	/api/announcements/:id	author's role scope, super_admin	Edit an existing announcement.
DELETE	/api/announcements/:id	super_admin only	Delete a published announcement.
GET	/api/events	All roles	List/search upcoming events.
POST	/api/events	principal, dean, hod, faculty, super_admin	Create a new event.
PUT	/api/events/:id	author's role scope, super_admin	Edit an existing event.
DELETE	/api/events/:id	super_admin only	Delete a published event.
GET	/api/alerts	All roles	List active/urgent alerts.
POST	/api/alerts	principal, dean, hod, faculty, super_admin	Create a new alert.

Method	Path	Role(s) Allowed	Description
PUT	/api/alerts/:id	author's role scope, super_admin	Edit an existing alert.
DELETE	/api/alerts/:id	super_admin only	Delete a published alert.
POST	/api/queries	student	Submit a new query.
GET	/api/queries	super_admin (all), hod (escalated), student (own)	List queries in scope.
PUT	/api/queries/:id/respond	super_admin, hod (escalated)	Respond to a query; updates status and responded_at.
POST	/api/webhooks/subscribe	super_admin	Register a downstream webhook subscriber.
GET	/* (non-API paths)	Public	Served by express.static("public") → React index.html.

7. Phase-Wise Development Plan

Eight phases, from discovery through post-launch support, each defined by what is actually built during that phase rather than by a separate documentation track.

Phase	What to Develop	Primary Deliverables
Phase 1 — Discovery & Requirements	Finalize the six-role RBAC model and the Super-Admin-only delete rule; lock the five-page scope; define functional/non-functional requirements; draft the initial database entity list.	Finalized requirements and role matrix.
Phase 2 — Design	Design the ERD & Postgres schema (users, announcements, events, alerts, queries, departments/faculties); design the full REST API contract (Section 6.3); create wireframes for the five pages plus the Super Admin dashboard; define the Tailwind design tokens (including the reserved red for Alerts); design the JWT + bcrypt auth flow and the role-authorization middleware.	DB schema, API contract, wireframes, security design.
Phase 3 — Backend Development	Scaffold the Express app per the folder structure in Section 6.1; implement bcrypt-based auth and JWT issuance; build the requireRole middleware; implement CRUD APIs for announcements, events, and alerts (create/edit open to publishing roles, delete locked to super_admin); build the Account Manager APIs (create/update/deactivate/reset-password, super_admin only); implement the Queries API with SLA timestamp tracking.	Working REST API, testable via Postman/curl.
Phase 4 — Frontend Development	Build the React + Tailwind app: Home, Announcements, Events, Alerts, and Contact pages; implement role-aware UI so create/edit controls render per role and the delete button renders only for the Super Admin; build the Super Admin dashboard (Account Manager UI); wire Axios/Fetch calls to the backend API.	Fully functional frontend running against the local/staging API.
Phase 5 — Real-Time & Integration	Implement the webhook dispatcher fired on announcement/alert publish events; add the live notice/event counters on the Home page; implement the	Live webhook endpoint; working real-time counters and countdown.

Phase	What to Develop	Primary Deliverables
	live countdown on the Alerts page (SSE or lightweight polling); wire optional downstream integrations (email digest, Slack/Teams relay).	
Phase 6 — Testing	Write Jest unit tests for the service layer (SLA calculation, role checks); write Supertest integration tests against a Neon test branch; run security tests — auth bypass, role escalation, and explicit verification that only super_admin can reach any DELETE route; run UAT with Principal, Dean, HoD, Faculty, and Student stakeholders.	Test reports and signed-off UAT.
Phase 7 — Deployment & Handover	Run npm run build in the frontend project; copy the dist output into the backend's public folder; remove any conflicting backend GET / route; confirm app.use(express.static("public")) and the catch-all fallback are in place; deploy the single combined backend project to Vercel; configure environment variables (Neon connection string, JWT secret, webhook URLs); set up uptime/error monitoring; distribute the Super Admin handbook and role-specific quick-start guides.	Live production system on a single deployment, monitoring in place, handbooks distributed.
Phase 8 — Post-Launch Support	Monitor error rates and webhook delivery; run a bug-fix cycle from early user feedback; review usage analytics (query SLA compliance, notice engagement); groom the backlog for v2 features (e.g., push notifications, richer analytics).	Support log and v2 backlog.

8. Code-Level Optimization Strategies

The strategies below reflect the recurring lessons that keep a system like NotifyHub fast, secure, and cheap to maintain as it scales from a single campus pilot to an institution-wide rollout, updated for the single-role Super Admin model and the combined single-deployment topology.

8.1 Frontend (React + Tailwind)

- Code-split by route (React.lazy/Suspense) so a Student loading the Announcements page never downloads the Super Admin dashboard bundle.
- Debounce the search input (e.g., 300ms) before firing API requests to avoid flooding the backend on every keystroke.
- Cache list responses client-side (e.g., with React Query/TanStack Query) with short stale times, so switching between filters doesn't always re-hit the network.
- Co-locate Tailwind utility classes with a small design-token layer (CSS variables for the ‘urgent red’) so the Alert color stays consistent and themeable.
- Virtualize long announcement/event lists (e.g., react-window) once notice volume grows into the thousands.
- Because the frontend and API now share the same origin (single combined deployment), there is no CORS overhead to manage — simplify Axios/Fetch calls accordingly.

8.2 Backend (Node.js + Express)

- Keep controllers thin: route → validate (e.g., Zod/Joi schema) → service layer → data-access layer. This separation keeps business logic testable independent of Express.

- Centralize the role-authorization check as reusable middleware (e.g., requireRole(['principal','dean','hod','faculty','super_admin'])), and treat every DELETE route as its own hard-coded permission class: requireRole(['super_admin']) with no other role ever added to that list.
- Use parameterized queries / an ORM (e.g., Prisma or Drizzle) against PostgreSQL to eliminate SQL-injection risk and to get compile-time query safety.
- Paginate every list endpoint (announcements, events, queries) by default — never return an unbounded result set.
- Apply rate limiting on authentication and query-submission endpoints to blunt brute-force and spam risks.
- Emit the webhook asynchronously (fire-and-forget with retry/backoff) so a slow downstream subscriber never blocks the API response to the publishing user.
- Mount app.use(express.static("public")) and the API routers before the catch-all '*' fallback that returns index.html, so an /api/* request is never accidentally served the frontend shell.
- Set long Cache-Control / immutable headers on the hashed, versioned frontend assets served from public (JS/CSS bundles), while keeping index.html itself uncached, so client updates propagate on the next visit.

8.3 Database (Neon PostgreSQL)

- Index foreign keys used in scoping (department_id, faculty_id) and any column used in filters (is_urgent, category, published_at) to keep search/filter queries fast.
- Use a full-text search index (PostgreSQL tsvector) for the notice search feature rather than naive LIKE '%term%' queries, which don't scale.
- Enforce data integrity with foreign-key constraints and CHECK constraints (e.g., the role enum, the query status enum) rather than relying solely on application-level validation.
- Leverage Neon's serverless branching for spinning up isolated database branches per pull request/preview deployment.

8.4 Security

- Hash every password with bcrypt at a cost factor appropriate to current hardware (e.g., 12) — never log or transmit plaintext passwords.
- Sign session tokens (JWT) with short expiry and use refresh tokens for longer sessions; store refresh tokens httpOnly and secure.
- Validate and sanitize all user-generated content (notice bodies, query messages) to prevent stored XSS, especially since notices are broadly viewed content.
- Apply the principle of least privilege: a Dean/HoD/Faculty JWT should carry their department/faculty scope so the backend can reject out-of-scope requests even if the frontend UI is bypassed.
- Guard every DELETE endpoint with an explicit, non-configurable requireRole(['super_admin']) check in addition to generic auth — deletion is the single most destructive action in the system and should be treated as its own permission class, not just another CRUD verb.
- Log every Account Manager action (creating, updating, deactivating an account, resetting a password) and every delete action to an append-only audit table.

8.5 Real-Time & Reliability

- Prefer a lightweight real-time channel (Server-Sent Events or a managed WebSocket provider) for the live notice counter and Alert countdown, reserving heavier infrastructure only if concurrent-connection counts justify it.
- Design the webhook contract with a versioned payload schema so downstream integrations don't break silently when the notice model evolves.
- Add idempotency keys to webhook deliveries and mutation endpoints so retried requests (network hiccups) don't create duplicate notices.

8.6 Testing & CI/CD

- Unit-test the service layer (business rules like SLA calculation, role checks) independent of HTTP and database concerns.
- Integration-test the Express routes against a disposable Neon test branch, including an explicit test that every non-super_admin role receives a 403 on every DELETE route.
- Run automated accessibility checks (axe-core) on the React pages, since students and staff will access NotifyHub on a range of devices.
- Wire GitHub Actions to run lint → unit tests → integration tests → build (frontend build + copy into backend/public) on every pull request, blocking merge on failure, before the combined project is promoted on Vercel.

9. Testing Strategy

9.1 Test Levels

Level	Focus	Tools/Approach
Unit	Service-layer business logic (SLA calc, role checks, validation rules)	Jest
Integration	Express routes against a real (test-branch) PostgreSQL database	Supertest + Neon test branch
End-to-End	Full user flows: login → post announcement → student searches → student raises query	Playwright/Cypress
Security	Auth bypass, role escalation, SQL injection, XSS, password hash verification, delete-route restriction	OWASP ZAP, manual pen-test checklist
Performance	Home page + notice list load time under simulated concurrent load	k6 / Lighthouse
UAT	Real Principal/Dean/HoD/Faculty/Student walkthroughs before go-live	Scripted scenarios, sign-off checklist

9.2 Key Non-Functional Test Scenarios

- Verify that no role other than super_admin can successfully call a DELETE endpoint for announcements, events, or alerts — including a direct, unauthenticated-UI API call.
- Verify a Principal, Dean, HoD, or Faculty account can create and edit content within scope but never sees or can trigger a delete action.

- Verify an Alert notice renders in red with a functioning live countdown across major browsers/devices.
- Verify a student query is timestamped correctly and the system flags it once it approaches the 6-hour SLA.
- Verify search/filter returns correct results across department and category combinations, including edge cases (no results, special characters).
- Verify webhook delivery retries on downstream failure and does not duplicate on retry (idempotency).
- Verify the combined deployment serves the frontend correctly: hitting “/” and any client-side route returns index.html, while /api/* requests are never intercepted by the static-file fallback.